

Contents

Twitter US Airlines — Module 3 Lab Guide	2
The Scenario — Your Mission	2
Where you work	2
The problem	2
Your manager's request	3
Your team's job for the next 2 weeks (Module 3)	3
After Module 3	3
A note about the dataset size	3
How This Guide Works (Read This First!)	4
We do NOT give you the full code	4
Visualizations — Two Modes	4
1. Exploratory charts (for YOU)	4
2. Explanatory charts (for JAMES)	4
Your plotting toolkit (you learned this in Module 2 Class 5)	4
Before You Start — Google Colab Setup	5
Step 1 — Open a new Colab notebook	5
Step 2 — Connect to Google Drive	5
Step 3 — Create a project folder	5
Step 4 — Get the data	5
Step 5 — Test it	6
Colab tips	6
Class 1 — Data Cleaning	6
Your goal	6
Inputs	6
Outputs	6
Phase A — Explore the data first (15 minutes)	6
Phase B — Clean the tweets table (45 minutes)	7
Phase C — Make ONE chart for James (15 minutes)	9
Common mistakes in Class 1	9
Self-check before Class 2	10
Class 2 — Encoding and Scaling	10
Your goal	10
Inputs	10
Outputs	10
Phase A — Explore the data first	10
Phase B — Encode and scale	11
Phase C — Make ONE chart for James	12
Common mistakes in Class 2	12
Self-check before Class 3	13
Class 3 — Feature Engineering	13
Your goal	13
Inputs	13
Outputs	13
Phase A — Explore the data first	13
Phase B — Engineer the features	14
Phase C — Make ONE chart for James	16
Common mistakes in Class 3	16
Self-check before Class 4	16

Class 4 — Feature Selection	16
Your goal	16
Inputs	17
Outputs	17
Phase A — Explore the data first	17
Phase B — Select features	17
Phase C — Make ONE chart for James	18
Common mistakes in Class 4	19
Self-check before Class 5	19
Class 5 — Pipelines	19
Your goal	19
Inputs	19
Outputs	19
Phase A — Explore the data first	19
Phase B — Build the pipeline	20
Phase C — Make ONE chart for James	21
Common mistakes in Class 5	21
Self-check before Class 6	22
Class 6 — End-to-End Lab (THE BIG ONE)	22
Your goal	22
Time	22
Inputs	22
Outputs	22
Phase A — Explore the data first (10 minutes)	22
Phase B — Build the final dataset (70 minutes)	22
Phase C — Findings report for James (10 minutes)	23
Grading rubric (100 points)	24
Submit	24
Tips for the whole module	25

Twitter US Airlines — Module 3 Lab Guide

Scenario: US Airlines on Twitter. Auto-classify customer service tweets. **Module:** 3 (Data Preparation and Feature Engineering) **Audience:** Adult learners, A2 English. New to AI. **Tools:** Google Colab (no install needed).

The Scenario — Your Mission

Where you work

You and your team are new data analysts at **American Airlines** in the United States. American Airlines is one of the biggest airlines in the world. They fly 200 million passengers every year.

Every day: - **10,000 tweets** are sent to airlines on Twitter. - People complain about late flights, lost bags, and bad food. - A few people say “thank you” for a kind flight attendant. - A team of 12 customer service agents reads the tweets.

The problem

The team of 12 can answer only **500 tweets per day**. That means **9,500 tweets get NO answer**.

What happens with those 9,500 tweets: - Angry customers tweet AGAIN, in capital letters. - Some send to TV news. - Some never fly American Airlines again. - Marketing misses the praise tweets and cannot share them. - Real urgent problems (lost baby stroller, missed medical flight) wait for hours.

The director is worried. He calls your team into a meeting on Monday morning.

Your manager's request

Your manager, **James** (Director of Customer Operations), tells you:

“We get 10,000 angry tweets a day. My team of 12 can answer 500.

Build a tool that reads each tweet and **routes it** to the right team: 1. ‘**Urgent complaint**’ → goes to a human agent right now. 2. ‘**Praise**’ → goes to marketing. They share it on TV. 3. ‘**Late flight**’ → goes to operations. They check the flight. 4. ‘**Lost luggage**’ → goes to the baggage team.

If we route correctly, we answer 10x more tweets with the same team.”

Your team's job for the next 2 weeks (Module 3)

James cannot do this alone. His data is **one big CSV file** with 14,600 tweets. The tweets have @mentions, hashtags, URLs, and many missing fields.

Your job in Module 3: > **Turn the messy CSV file into ONE clean file. The clean file will be used to train the routing model in Module 4.**

The clean file is called `airline_tweets_clean.parquet`. It must have **19 specific columns** (we will see them in Class 6).

After Module 3

Module	What happens
Module 4	Train the routing model. James gets his “tweet category” prediction.
Module 5	Find groups of customers (super angry, neutral, fans). For marketing.
Module 7	Read the tweet text itself. Find common complaint topics.

You use the **same Twitter Airlines dataset** until the end of Module 7.

A note about the dataset size

This dataset has only **~14,600 rows**. That is small. The Olist dataset had 99,000. Some teams will be worried. **Do not worry.**

Small data is GOOD for learning, because: - Every step runs fast (no waiting). - You can SEE every change you make. - Real customer service teams also have small data when they start. - Module 4 will show you tricks for small data (cross-validation, regularization).

In a real airline, the dataset would grow to millions of tweets. But the steps you do today are exactly the same.

How This Guide Works (Read This First!)

We do NOT give you the full code

In this guide you will see two kinds of content:

1. WHAT / WHY / EXPECTED — explained in words

For every step you read: - **WHAT** you have to do (in normal language) - **WHY** you do it (the reason) - **HINTS** — the function names, the arguments to remember - **EXPECTED** — the result you should see if you did it right

2. Sometimes a tiny code skeleton with blanks

For the harder steps we give you a code skeleton with ____ blanks. **You fill in the blanks.**

Why no full code?

Because you will NOT learn if you copy-paste. - In Module 4, the model only works if YOU understand each step. - In your future job, nobody will give you the code. - Writing the code yourself takes 10 minutes more today, but saves 10 hours later.

Rule: If you finish a step in 30 seconds by copy-pasting from a teammate's notebook, do it AGAIN by yourself. Different variable names, same logic.

Visualizations — Two Modes

In every class you will make charts. There are **two reasons** to make a chart:

1. Exploratory charts (for YOU)

Made BEFORE you clean the data. To understand what the data looks like. - Fast, ugly, no labels needed. - Examples: `df.hist()`, `df['column'].value_counts().plot.bar()`.

Goal: “**What does this data look like?**”

2. Explanatory charts (for JAMES)

Made AFTER you understand. To EXPLAIN something to your manager. - Clean, labeled, one clear message. - Must have: title, x-label, y-label, source, and a 1-sentence takeaway.

Goal: “**James, look at this. This is the problem.**”

In every class you make BOTH kinds.

Your plotting toolkit (you learned this in Module 2 Class 5)

Plot	When to use it	Function name
Histogram	See the SHAPE of a numeric column	<code>plt.hist()</code> or <code>sns.histplot()</code>
Bar chart	Count of each category	<code>df['col'].value_counts().plot.bar()</code>
Box plot	See outliers in a numeric column	<code>sns.boxplot()</code>
Scatter plot	See the relationship between 2 numbers	<code>plt.scatter()</code> or <code>sns.scatterplot()</code>
Heatmap	See correlation between many columns	<code>sns.heatmap(df.corr())</code>

Plot	When to use it	Function name
Line plot	See change over time	<code>plt.plot()</code>

Before you start: `import matplotlib.pyplot as plt` and `import seaborn as sns`.

Before You Start — Google Colab Setup

You will work in **Google Colab**. That means: - You do NOT install Python. - You do NOT install pandas, scikit-learn, or anything. - You just open a notebook in your web browser.

Step 1 — Open a new Colab notebook

1. Go to <https://colab.research.google.com>
2. Sign in with your Google account.
3. Click “**New notebook**” (top left).
4. Name it `airline_tweets_module_3.ipynb`.

Step 2 — Connect to Google Drive

Colab files **disappear** when you close the browser. So you must save your work to **Google Drive**.

In the first cell:

```
from google.colab import drive
drive.mount('/content/drive')
```

A pop-up asks for permission. Click “Allow”. After this, Drive is available at `/content/drive/MyDrive/`.

Step 3 — Create a project folder

In a new cell:

```
import os
os.makedirs('/content/drive/MyDrive/airline_lab', exist_ok=True)
%cd /content/drive/MyDrive/airline_lab
```

Your team will save ALL files in this folder.

Step 4 — Get the data

Option A — Direct from Kaggle (recommended):

1. Make a free Kaggle account.
2. Go to kaggle.com/settings/account, scroll to “API”, click “Create New Token”. Save the `kaggle.json` file.
3. In Colab:

```
from google.colab import files
files.upload() # pick kaggle.json
```

4. Then:

```
!mkdir -p ~/.kaggle && mv kaggle.json ~/.kaggle/ && chmod 600 ~/.kaggle/kaggle.json
!pip install kaggle -q
!kaggle datasets download -d crowdflower/twitter-airline-sentiment
!unzip -q twitter-airline-sentiment.zip -d data
!ls data/
```

You should see **1 CSV file** inside data/ (plus maybe a database file you can ignore).

Option B — Upload by hand:

1. Download the zip from [kaggle.com/datasets/crowdflower/twitter-airline-sentiment](https://www.kaggle.com/datasets/crowdflower/twitter-airline-sentiment) on your laptop.
2. Unzip it.
3. In Colab's file panel (left sidebar), upload the CSV file.

Step 5 — Test it

```
import pandas as pd
tweets = pd.read_csv('data/Tweets.csv')
print(tweets.shape)
```

Should print (14640, 15). You are ready.

Colab tips

Tip	Why
Save your notebook to Drive often	Colab disconnects after 12 hours.
Save intermediate .parquet files to Drive	Files in /content/ disappear when Colab disconnects.
Share the notebook with teammates (Share button, top right)	Editor access. Like Google Docs for code.
Restart runtime if memory full	Runtime > Restart runtime. (You will rarely need this with only 14k rows.)

Class 1 — Data Cleaning

Scenario reminder: James drops the messy CSV on your desk. The tweet timestamp is stored as text. About half the rows have no timezone. Some rows have URLs and @mentions inside the text. Your job today: load the file, fix the date column, decide what to drop.

Your goal

Make the CSV file **USABLE**. Fix the date column. Find missing values. Decide what to keep.

Inputs

- The CSV file `Tweets.csv` in data/

Outputs

- `tweets_step1.parquet` saved in your Drive folder
- 3+ exploratory charts in your notebook
- 1 explanatory chart for James
- Notes in markdown about every decision you made

Phase A — Explore the data first (15 minutes)

Before you clean anything, **LOOK** at the data.

Exploratory chart 1 — How many tweets per sentiment?

- **Question:** “Of 14,640 tweets, how many are negative, neutral, positive?”
- **HINTS:**
 - Use `tweets['airline_sentiment'].value_counts()`.
 - Then `.plot.bar()` on the result.
- **What you learn:** Most tweets are negative (~63%). The classes are NOT balanced. Module 4 will need to handle this.

Exploratory chart 2 — How many tweets per airline?

- **Question:** “Which airline gets the most tweets?”
- **HINTS:**
 - Use `tweets['airline'].value_counts()`.
 - Plot as a bar chart.
- **What you learn:** 6 airlines. United and US Airways get the most. Some get few. This may affect the model.

Exploratory chart 3 — Missing values per column

- **Question:** “Which columns have the most missing data?”
- **HINTS:**
 - Use `tweets.isna().sum().sort_values(ascending=False)`.
 - Plot it as a bar chart.
- **What you learn:** `tweet_coord` is missing in almost every row. `negativereason` is missing in positive and neutral tweets. `user_timezone` is missing about half the time.

Exploratory chart 4 — Why are people negative?

- **Question:** “When a tweet is negative, what is the reason?”
 - **HINTS:**
 - Filter to negative tweets only.
 - `tweets[tweets['airline_sentiment']=='negative']['negativereason'].value_counts().plot.bar()`
 - **What you learn:** The top reasons (customer service issue, late flight, can't tell, cancelled flight, lost luggage). This is the routing list James asked for.
-

Phase B — Clean the tweets table (45 minutes)

Step 1 — Load the CSV file

- **WHAT:** Load the CSV file into a DataFrame called `tweets`.
- **HINTS:**
 - Use `pd.read_csv('data/Tweets.csv')`.
 - Some columns may need a specific encoding. If you see strange characters, try `encoding='utf-8'` or `encoding='latin-1'`.
- **EXPECTED:** A DataFrame with 14,640 rows and 15 columns.

Step 2 — Look at the DataFrame

- **WHAT:** Check `.shape`, `.info()`, and `.head()`.
- **HINTS:**
 - Look at the `Dtype` column in `.info()` output. **Is `tweet_created` stored as object (text)?**
 - Look at the first 5 rows with `.head()`. The text column has @VirginAmerica mentions and URLs. Note this for later.
- **EXPECTED:**

Column group	Examples
Target	airline_sentiment, airline_sentiment_confidence
Routing	negativereason, negativereason_confidence
Context	airline, name, user_timezone
Timestamps	tweet_created (text — needs fixing)
Text	text (the tweet itself)
Location	tweet_coord, tweet_location (mostly missing)
Other	retweet_count, tweet_id

Step 3 — Fix the date column

- **WHAT:** The `tweet_created` column is stored as text like "2015-02-24 11:35:52 -0800". Convert it to a real datetime.
- **HINTS:**
 - The function is `pd.to_datetime()`.
 - Add the argument `errors='coerce'`. If a cell is bad, it becomes NaT (Not a Time = missing). The code does not crash.
 - The string has a timezone (the `-0800` part). Add the argument `utc=True` to convert everything to UTC. This makes all dates comparable.
- **WHY:** If dates are strings, you cannot extract the hour or the day of week. "What time of day do most complaints happen?" is impossible without a real datetime.
- **EXPECTED:** After your code, `tweets['tweet_created'].dtype` shows `datetime64[ns, UTC]`.

Step 4 — Find missing values

- **WHAT:** Count missing values per column.
- **HINTS:** `.isna()` returns True/False per cell. `.sum()` counts the Trues per column.
- **EXPECTED:** Something like:

```
tweet_coord          13621
negativereason_gold  14608
airline_sentiment_gold 14600
negativereason       5462
user_timezone        4820
tweet_location       4733
```

Step 5 — Drop the useless columns

- **WHAT:** Some columns have almost no data, or will not help the model.
- **DROP these columns:**
 - `tweet_coord` — ~93% missing.
 - `airline_sentiment_gold` — ~99% missing (only used by data labelers).
 - `negativereason_gold` — ~99% missing.
 - `tweet_location` — too free-text, ~30% missing.
- **HINTS:**
 - Use `.drop(columns=[...])`.
 - Or pass a list to `tweets = tweets.drop(['col1', 'col2'], axis=1)`.
- **WHY:** If a column is empty 93% of the time, the model cannot use it. Better to remove it now and keep the table clean.

Step 6 — Decide: drop rows with no text

- **WHAT:** Check if any row has missing text. If yes, drop it.
- **HINTS:**
 - `tweets['text'].isna().sum()` → probably 0, but always check.

- If > 0 , use `.dropna(subset=['text'])`.
- **WHY:** The whole point of this dataset is the text. No text = useless row.
- **EXPECTED:** Probably still 14,640 rows.

Step 7 — Check for duplicate tweets

- **WHAT:** Are there any exact duplicate rows?
- **HINTS:**
 - Use `tweets.duplicated().sum()`.
 - If > 0 , use `tweets.drop_duplicates()`.
- **WHY:** Duplicates would bias the model.
- **EXPECTED:** Probably very few duplicates (less than 50).

Step 8 — Write down what you did

In a markdown cell, write: - Starting rows: 14,640 - Starting columns: 15 - Columns dropped (and why): `tweet_coord`, `airline_sentiment_gold`, `negativereason_gold`, `tweet_location` - Final rows: ~14,600 - Final columns: 11

Step 9 — Save to Drive

- **WHAT:** Save the cleaned tweets DataFrame as parquet, inside your Drive folder.
- **HINTS:** `df.to_parquet('/content/drive/MyDrive/airline_lab/tweets_step1.parquet')`.

Phase C — Make ONE chart for James (15 minutes)

He does not have time to read your code. He wants ONE picture.

James's chart — “Complaint volume by hour of day”

Make a bar chart showing how many tweets arrive in each hour of the day (0–23). Use UTC time for now.

- **HINTS:**
 - First extract the hour: `tweets['tweet_hour'] = tweets['tweet_created'].dt.hour`.
 - Group: `tweets.groupby('tweet_hour').size()`.
 - Plot as a bar chart.
 - Title: “Tweet volume by hour of day — peak hours need more agents”.
 - X-label: Hour of day (UTC).
 - Y-label: Number of tweets.
- **Takeaway for James:** “Most tweets come between 14:00 and 22:00 UTC. We should have more agents in those hours.”

Common mistakes in Class 1

Mistake	What goes wrong
Forget <code>errors='coerce'</code> on <code>to_datetime</code>	Code crashes on one bad date.
Forget <code>utc=True</code> on <code>to_datetime</code>	Some dates have offsets, some do not. Comparing them fails.
Drop <code>negativereason</code> because it has missing values	Big mistake. It is missing for positive/neutral tweets, which is normal.
Save to <code>/content/</code> instead of Drive	File disappears when Colab disconnects.

Self-check before Class 2

- ☐ CSV loaded. 14,640 rows.
 - ☐ `tweet_created` has dtype `datetime64[ns, UTC]`.
 - ☐ 4 useless columns dropped.
 - ☐ At least 3 exploratory charts in your notebook.
 - ☐ 1 polished chart for James.
 - ☐ You saved `tweets_step1.parquet` to your Drive folder.
 - ☐ You wrote down (in markdown) what you did and WHY.
-

Class 2 — Encoding and Scaling

Scenario reminder: James looks at your cleaned data. He is happy. But he says: “the model is a math model. It does not understand the word ‘United’ or the word ‘negative’. Turn the words into numbers.”

Your goal

Turn TEXT columns into numbers. Make all numeric columns about the same size.

Inputs

- `tweets_step1.parquet` from Class 1

Outputs

- `tweets_step2.parquet` in Drive
 - 3+ exploratory charts
 - 1 explanatory chart for James
-

Phase A — Explore the data first

Exploratory chart 1 — Distribution of `airline_sentiment_confidence`

- **Question:** “The labelers said how confident they were. What does this look like?”
- **HINTS:**
 - Use a histogram (`plt.hist()`).
 - Use `bins=30` to see the shape.
- **What you learn:** Many tweets have confidence 1.0 (sure). A few have confidence below 0.5 (the labelers were unsure). Are there tweets where the label is bad?

Exploratory chart 2 — Distribution of `retweet_count`

- **Question:** “Most tweets have 0 retweets. A few have 30+. Is the distribution skewed?”
- **HINTS:**
 - Use a histogram.
 - Use `bins=50`.
 - Use `plt.yscale('log')` to see the small values too.
- **What you learn:** The retweet column has a “long tail”. This is why we will use log-transform.

Exploratory chart 3 — airline counts after combining

- **Question:** “Which airline has the most tweets? Should we keep all 6?”
- **HINTS:** `value_counts().plot.bar()`.
- **What you learn:** 6 airlines. United, US Airways, American get the most. Virgin America has only a few hundred.

Exploratory chart 4 — negativereason counts

- **Question:** “Of the 10 negative reasons, which are the biggest?”
 - **HINTS:**
 - Filter to negative tweets first.
 - `value_counts().plot.bar()`.
 - **What you learn:** “Customer Service Issue” and “Late Flight” are the biggest. “Damaged Luggage” is rare. This affects routing.
-

Phase B — Encode and scale

Step 1 — Load the cleaned data

- **WHAT:** Load `tweets_step1.parquet` from your Drive folder.
- **HINTS:** `tweets = pd.read_parquet('/content/drive/MyDrive/airline_lab/tweets_step1.parquet')`.

Step 2 — One-hot encode airline

- **WHAT:** Turn the 6 airlines into 6 new 0/1 columns.
- **HINTS:**
 - Use `pd.get_dummies(tweets, columns=['airline'], prefix='airline')`.
- **WHY:** The model is math. “United” is text. After one-hot, you get 6 new columns: `airline_United = 1` or `0`, `airline_Delta = 1` or `0`, etc.
- **EXPECTED:** 6 new columns added.

Step 3 — Decide what to do with negativereason

- **WHAT:** This column has 10 categories AND about 5,500 missing values (for positive/neutral tweets).
- **TWO OPTIONS:**
 - **A — Fill missing with 'not_negative':** Then one-hot encode. You get 11 categories.
 - **B — Keep missing as a separate category with `pd.get_dummies(..., dummy_na=True)`:** Pandas adds a “missing” column automatically.
- **HINTS:**
 - Option A: `tweets['negativereason'] = tweets['negativereason'].fillna('not_negative')`.
 - Then `pd.get_dummies(tweets, columns=['negativereason'], prefix='reason')`.
- **YOUR CHOICE:** Pick one. Write in your notebook WHY.

Step 4 — Encode the target airline_sentiment

- **WHAT:** This is the column the model will PREDICT. Turn the 3 categories (negative, neutral, positive) into numbers.
- **HINTS:**
 - You can use `LabelEncoder` from `sklearn`.
 - Or a simple map: `tweets['airline_sentiment'] = tweets['airline_sentiment'].map({'negative': 0, 'neutral': 1, 'positive': 2})`.
- **WHY a map and not get_dummies?** For the TARGET, we want one column with values 0/1/2. Not 3 columns of 0/1.

Step 5 — Log-transform retweet_count

- **WHAT:** Most tweets have 0 retweets. A few have 30+. Apply `np.log1p()`.
- **HINTS:**
 - `import numpy as np.`
 - `tweets['log_retweet'] = np.log1p(tweets['retweet_count'])`.
- **WHY log1p and not log?** `log(0)` is `-inf`. `log1p(x) = log(x + 1)` is safe for zeros.
- **WHY transform at all?** A column with values 0, 1, 30 is hard for the model. After log: 0, 0.69, 3.4. Easier.

Step 6 — Scale numeric columns (preview only)

- **WHAT:** Use `StandardScaler` to make every numeric column have mean = 0, std = 1.
- **HINTS:**
 - `from sklearn.preprocessing import StandardScaler.`
 - `scaler = StandardScaler().`
 - `scaler.fit_transform(tweets[numeric_cols])`.
- **WARNING:** This is for inspection only. In Class 5, scaling goes INSIDE the Pipeline. Do NOT save the scaled version as your final file.

Step 7 — Decide what to do with user_timezone

- **WHAT:** About half the rows have NO timezone. The other half have ~80 different timezones.
- **OPTIONS:**
 - Drop the column.
 - Keep only the top 5 timezones, group the rest as “other”, then one-hot encode.
- **HINTS:**
 - Top 5: `top5 = tweets['user_timezone'].value_counts().head(5).index.`
 - Group: `tweets['user_timezone'] = tweets['user_timezone'].where(tweets['user_timezone'].isin('other')).`
- **WRITE DOWN:** Your choice in markdown.

Step 8 — Save

- **WHAT:** Save to Drive. `df.to_parquet('/content/drive/MyDrive/airline_lab/tweets_step2.parquet')`.
 - **Save the UNSCALED version.** Scaling will happen inside the Pipeline later.
-

Phase C — Make ONE chart for James

James’s chart — “Tweet count by airline”

A bar chart of all 6 airlines, ordered by tweet count.

- **HINTS:**
 - `tweets['airline'].value_counts().plot.bar()`. (Use the ORIGINAL column before one-hot.)
 - **Title:** “Tweet volume by airline — United and US Airways get half of all tweets.”
 - **X-label:** Airline.
 - **Y-label:** Number of tweets.
 - **Takeaway:** “Two airlines drive most of the volume. Train the model on data from all 6, but expect best performance on these two.”
-

Common mistakes in Class 2

Mistake	What goes wrong
Scale BEFORE train/test split One-hot encode every text column	Leakage. Scaler sees test data. The text column has 14,000 different values. Table explodes. Leave it for Module 7.
Forget <code>np.log1p</code> and use <code>np.log</code> on zeros Save the scaled version	<code>np.log(0) = -inf</code> . Crash. Class 5 will scale again inside the pipeline. Double scaling = wrong numbers.
Drop <code>negativereason</code> because it has 5500 missing	Big mistake. The missing means “not a negative tweet”. Treat it as a category.

Self-check before Class 3

- ☐ `airline` one-hot encoded (6 new columns).
- ☐ `negativereason` handled (your choice written down).
- ☐ `airline_sentiment` mapped to 0/1/2.
- ☐ `log_retweet` exists.
- ☐ At least 3 exploratory charts.
- ☐ 1 chart for James.
- ☐ `tweets_step2.parquet` saved to Drive.

Class 3 — Feature Engineering

Scenario reminder: James says: “The columns in the raw data are not enough. The TRULY useful columns are not there. We must MAKE them. For example, how many @mentions does the tweet have? How many capital words? Are there exclamation marks? Those small signals tell us if a tweet is urgent.”

Your goal

Make NEW columns from the existing ones. These will help the model predict the sentiment and the routing class.

Inputs

- `tweets_step2.parquet`

Outputs

- `tweets_step3.parquet` in Drive
- 3+ exploratory charts
- 1 explanatory chart for James

Phase A — Explore the data first

Exploratory chart 1 — Tweet length distribution

- First make the `text_length` column (see Step 2 below).
- **Question:** “How long is a typical tweet?”
- **HINTS:** Histogram with `bins=50`.
- **What you learn:** Twitter limits tweets to 140 characters (in 2015). You will see most tweets near the limit. Angry people use more characters.

Exploratory chart 2 — Sentiment by hour of day

- **Question:** “Are tweets more negative at night? Or in the morning?”
- **HINTS:**
 - Use `tweets.groupby('tweet_hour')['airline_sentiment'].value_counts(normalize=True)`.
 - Or convert sentiment to a number (`neg=-1, neu=0, pos=1`) and take the mean per hour.
 - Plot as a line or bar chart.
- **What you learn:** Patterns by hour. Maybe night flights are worst.

Exploratory chart 3 — Tweet length vs sentiment

- **Question:** “Do negative tweets have more characters than positive ones?”
- **HINTS:**
 - Use `sns.boxplot(x='airline_sentiment', y='text_length', data=tweets)`.
- **What you learn:** Angry tweets are usually LONGER. People type more when they are upset.

Exploratory chart 4 — Capital words vs sentiment

- After you compute `n_caps_words` (Step 4 below), look at it by sentiment.
- **HINTS:** Boxplot or bar chart of mean per sentiment class.
- **What you learn:** “WORST FLIGHT EVER!!!” has 3 caps words. Polite tweets have 0. Big signal.

Phase B — Engineer the features

Step 1 — Load the data

- **HINTS:** `tweets = pd.read_parquet('/content/drive/MyDrive/airline_lab/tweets_step2.parquet')`.

Step 2 — Date-derived features

Make these new columns from `tweet_created`:

New column	What it is
<code>tweet_year</code>	The year
<code>tweet_month</code>	The month (1–12)
<code>tweet_dayofweek</code>	0=Monday, 6=Sunday
<code>tweet_hour</code>	0–23
<code>is_weekend</code>	1 if <code>dayofweek >= 5</code> , else 0

- **HINTS:**
 - Use the `.dt` accessor on the datetime column.
 - `tweets['tweet_created'].dt.year, .dt.month, .dt.dayofweek, .dt.hour`.
 - For `is_weekend`, write a comparison and convert to int with `.astype(int)`.
- **WHY:** A model can learn “Sunday-night tweets are angrier” only if YOU give it the `tweet_dayofweek` column.
- **NOTE:** All tweets in this dataset are from February 2015. `tweet_year` and `tweet_month` will be almost constant. Keep them anyway, for completeness. They will be dropped in Class 4 if useless.

Step 3 — Text length features

New column	What it is
<code>text_length</code>	Number of characters in the tweet

- **HINTS:**

- tweets['text_length'] = tweets['text'].str.len().
- **WHY:** Longer tweets often mean angrier customers.

Step 4 — Text content features

These are very important for tweet classification.

New column	What it is
n_mentions	Number of @mentions in the tweet
n_hashtags	Number of #hashtags in the tweet
has_url	1 if the tweet has a link, else 0
n_caps_words	Number of fully UPPERCASE words (length > 1)
has_question_mark	1 if the tweet has at least one ?, else 0
has_exclamation	1 if the tweet has at least one !, else 0

- **HINTS:**
 - For n_mentions: tweets['text'].str.count('@').
 - For n_hashtags: tweets['text'].str.count('#').
 - For has_url: check if the text contains http. tweets['text'].str.contains('http').astype(int).
 - For n_caps_words: split the text into words, count words where word.isupper() and len(word) > 1. You can write a small function and apply it.
 - For has_question_mark: tweets['text'].str.contains(r'\?').astype(int) (the \? is regex for literal ?).
 - For has_exclamation: tweets['text'].str.contains('!').astype(int).
- **WHY:** Tweet text is short. These small features capture “shouting” signals.

Step 5 — Build a simple sentiment lexicon score

This is an EXTRA feature that helps the model a lot.

- **WHAT:** Make a small list of positive words and negative words. For each tweet, count positives minus negatives.

Skeleton:

```
positive_words = ['thank', 'great', 'love', 'awesome', 'amazing', 'best', '___'] # YOU add 5 more
negative_words = ['worst', 'terrible', 'late', 'lost', 'cancelled', 'awful', '___'] # YOU add 5 more
```

```
def lexicon_score(text):
    text_lower = text.__( ) # YOU fill in (the lowercase method)
    pos_count = sum(w in text_lower for w in __ ) # YOU fill in
    neg_count = sum(w in text_lower for w in __ ) # YOU fill in
    return pos_count - neg_count
```

```
tweets['sentiment_lexicon_score'] = tweets['text'].apply( ) # YOU fill in
```

- **WHY:** This single number already captures a lot. Module 7 will do this more carefully with a real NLP library.
- **EXPECTED:** sentiment_lexicon_score between -5 and +5 for most tweets.

Step 6 — Sanity-check the new columns

- **WHAT:** For each new column, look at .describe() and check the range is reasonable.
- **HINTS:**
 - tweets[['text_length', 'n_mentions', 'n_hashtags', 'n_caps_words']].describe().
- **EXPECTED:**
 - text_length: 0 to 200 (tweets are short).
 - n_mentions: 0 to maybe 5.
 - n_hashtags: 0 to maybe 3.
 - n_caps_words: 0 to maybe 10.

Step 7 — Save

- **HINTS:** `tweets.to_parquet('/content/drive/MyDrive/airline_lab/tweets_step3.parquet')`.
-

Phase C — Make ONE chart for James

James's chart — “What makes a tweet angry? Three signals.”

A grouped bar chart showing the MEAN of 3 features for each sentiment class.

- **HINTS:**
 - Compute: `tweets.groupby('airline_sentiment')[['text_length', 'n_caps_words', 'has_exclamation']].mean()`.
 - Plot as a grouped bar chart with 3 groups (negative, neutral, positive) and 3 bars per group.
 - **Title:** “Negative tweets are longer, louder, and full of exclamation marks.”
 - **X-axis:** Sentiment.
 - **Y-axis:** Mean value of the feature.
 - **Takeaway for James:** “If a tweet has 5+ caps words and 3 exclamation marks, it is almost always angry. We can route those to a human agent in 1 second.”
-

Common mistakes in Class 3

Mistake	What goes wrong
Use the FULL text as a feature	The model cannot use raw text directly. That is Module 7.
Forget to lowercase before lexicon matching	“Thank” and “thank” become different. Misses half the matches.
Use <code>str.contains('http')</code> with <code>regex=False</code> missing	Default is <code>regex=True</code> , which mostly works, but special characters in the pattern can cause bugs. Use simple patterns.
Count @ as <code>str.contains('@')</code> instead of <code>str.count('@')</code>	<code>contains</code> returns True/False (1 or 0). <code>count</code> returns the actual number. We want the count.

Self-check before Class 4

- ☐ 5 date-derived features exist.
 - ☐ `text_length` exists.
 - ☐ `n_mentions`, `n_hashtags`, `has_url`, `n_caps_words`, `has_question_mark`, `has_exclamation` all exist.
 - ☐ `sentiment_lexicon_score` exists.
 - ☐ 3+ exploratory charts.
 - ☐ 1 explanatory chart.
 - ☐ `tweets_step3.parquet` saved to Drive.
-

Class 4 — Feature Selection

Scenario reminder: Now you have ~30 columns. James says: “Too many. Some are duplicates. Some are useless. I want 12–15 GOOD columns. Pick them.”

Your goal

Pick the best 12–15 columns. Drop the rest. Justify every choice.

Inputs

- tweets_step3.parquet

Outputs

- tweets_step4.parquet in Drive (only the selected columns + airline_sentiment)
 - 3+ exploratory charts (correlation heatmap, mutual info bars, importance bars)
 - A short markdown report explaining your choices
-

Phase A — Explore the data first

Exploratory chart 1 — Correlation heatmap of numeric columns

- **Question:** “Which columns say the same thing as each other?”
- **HINTS:**
 - Compute `tweets[numeric_cols].corr()`.
 - Plot with `sns.heatmap()`.
 - Use `annot=True` and `fmt='.2f'`.
- **What you learn:** Pairs of columns with `lccorr > 0.9` are redundant. For example, `text_length` and `n_mentions` may be correlated.

Exploratory chart 2 — Mutual information bar chart

- After you compute mutual info (Step 4 below), plot it.
- **HINTS:**
 - Sort the values, then bar chart.
- **What you learn:** Which columns predict `airline_sentiment` strongest.

Exploratory chart 3 — Random Forest feature importance

- After you train the RF (Step 5 below), plot the importances.
 - **HINTS:** `pd.Series(rf.feature_importances_, index=...).sort_values().plot.barh()`.
 - **What you learn:** A second opinion on feature importance.
-

Phase B — Select features

Step 1 — Split into train and test FIRST

- **WHAT:** Use `train_test_split`.
- **HINTS:**
 - from `sklearn.model_selection` import `train_test_split`.
 - `X = tweets.drop('airline_sentiment', axis=1)`.
 - `y = tweets['airline_sentiment']`.
 - Pass: `test_size=0.2, random_state=42, stratify=y`.
- **WHY stratify=y?** Only ~16% of tweets are positive. Without stratify, the test set might have 5% positive by accident. Stratify keeps the same balance.
- **WHY split first?** All next steps use TRAIN ONLY. Otherwise leakage.

Step 2 — Remove low-variance columns

- **WHAT:** Drop columns where almost all values are the same.
- **HINTS:**
 - from `sklearn.feature_selection` import `VarianceThreshold`.

- `VarianceThreshold(threshold=0.01)` then `.fit_transform(X_train[numeric_cols])`.
- **EXPECTED:** `tweet_year` may be dropped (all tweets are from 2015).

Step 3 — Remove highly correlated columns

- **WHAT:** Drop one of each pair where `lcorr > 0.9`.
- **HINTS:**
 - Compute correlation matrix.
 - Get the upper triangle with `np.triu(...)`.
 - For each column, if any of its upper-triangle correlations is `> 0.9`, mark it for dropping.

Step 4 — Rank by mutual information

- **WHAT:** Score each column by how much it tells you about `airline_sentiment`.
- **HINTS:**
 - `from sklearn.feature_selection import mutual_info_classif`.
 - `mi = mutual_info_classif(X_train_numeric, y_train)`.
 - Put in a Series, sort.
- **EXPECTED:** `sentiment_lexicon_score`, `n_caps_words`, `has_exclamation`, `text_length` should be at the top.

Step 5 — Random Forest importance (second opinion)

- **WHAT:** Train a small RF, look at `feature_importances_`.
- **HINTS:**
 - `from sklearn.ensemble import RandomForestClassifier`.
 - `RandomForestClassifier(n_estimators=100, max_depth=10, n_jobs=-1, random_state=42)`.
 - `.fit(X_train, y_train)`.
 - Look at `.feature_importances_`.

Step 6 — Pick the final 12–15 columns

- **WHAT:** Combine the rankings. Pick columns that:
 - Are high in mutual info, AND
 - Are high in RF importance, AND
 - Were not dropped by variance/correlation pruning.
- **WRITE DOWN:** In your notebook, list the columns. Explain in 1 sentence why each one is in.

Step 7 — Save

- **HINTS:** Keep only the selected columns + `airline_sentiment`. Save as `tweets_step4.parquet` to Drive.

Phase C — Make ONE chart for James

James's chart — “The 10 strongest signals for tweet sentiment”

A horizontal bar chart of your top 10 features and their importance score.

- **HINTS:**
 - Use the RF importance.
 - `sort_values()` then `.head(10)` then `.plot.barh()`.
- **Title:** “Top 10 predictors of tweet sentiment.”
- **Y-axis:** column names.
- **X-axis:** Random Forest importance.

- **Takeaway:** “The lexicon score and ‘caps words’ alone explain ~50% of the prediction. Easy signals. We can use a simple model.”

Common mistakes in Class 4

Mistake	What goes wrong
Compute mutual info on the FULL data	Leakage.
Drop columns without writing why	Module 4 students will not understand.
Keep too many columns (40+)	Slow training. Overfitting risk. Small dataset (14k) cannot support many columns.
Drop <code>airline_sentiment_confidence</code> because it sounds technical	This is useful. It tells you how clean the label is. Keep it.

Self-check before Class 5

- ☐ Train/test split done FIRST.
- ☐ 12–15 columns remain + `airline_sentiment`.
- ☐ You wrote down WHY for each kept column.
- ☐ 3+ exploratory charts.
- ☐ 1 chart for James.
- ☐ `tweets_step4.parquet` saved to Drive.

Class 5 — Pipelines

Scenario reminder: James says: “Your cleaning code is in 4 different notebooks. When a new tweet arrives in 10 seconds, you cannot run 4 notebooks. We need ONE object that does everything.”

Your goal

Put EVERY cleaning step inside ONE Pipeline object. Production-ready code.

Inputs

- `tweets_step4.parquet` (selected columns)

Outputs

- `airline_pipeline.joblib` saved in Drive
- 1 confusion-matrix chart + 1 classification report
- A pipeline that takes RAW input and produces predictions

Phase A — Explore the data first

Exploratory chart 1 — Confusion matrix (after training)

- After Step 6, build a confusion matrix.
- **HINTS:**
 - `from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay.`

- `cm = confusion_matrix(y_test, y_pred).`
- `ConfusionMatrixDisplay(cm, display_labels=['Negative', 'Neutral', 'Positive']).plot().`
- **What you learn:** Does the model confuse neutral and positive? Does it catch the negative tweets (most important)?

Exploratory chart 2 — Per-class precision and recall

- **HINTS:**
 - `from sklearn.metrics import classification_report.`
 - `print(classification_report(y_test, y_pred, target_names=['Negative', 'Neutral', 'Positive'])).`
 - **What you learn:** Which class is hardest? Probably “neutral” — it is the middle.
-

Phase B — Build the pipeline

Step 1 — Decide which columns are numeric and which are categorical

- **HINTS:** Make two lists.
- **EXAMPLE:**
 - numeric: `text_length, n_mentions, n_hashtags, n_caps_words, sentiment_lexicon_score, tweet_hour, is_weekend, airline_sentiment_confidence, log_retweet, has_url, has_question_mark, has_exclamation.`
 - categorical: `airline, negativereason` (if you kept it as a string).

Step 2 — Build the numeric mini-pipeline

- **WHAT:** 2 steps: imputer (fill missing) + scaler.
- **HINTS:**
 - `from sklearn.pipeline import Pipeline.`
 - `from sklearn.impute import SimpleImputer.`
 - `from sklearn.preprocessing import StandardScaler.`
 - Skeleton:


```
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='___')),
    ('scaler', StandardScaler()),
])
```
 - Fill in the blank: best strategy for numeric? ('median')

Step 3 — Build the categorical mini-pipeline

- **WHAT:** 2 steps: imputer + one-hot encoder.
- **HINTS:**
 - `from sklearn.preprocessing import OneHotEncoder.`
 - Skeleton:


```
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='___')),
    ('onehot', OneHotEncoder(handle_unknown='___', sparse_output=___)),
])
```
 - Fill in blanks. (Strategy 'most_frequent', handle_unknown 'ignore', sparse_output False.)
- **WHY handle_unknown='ignore'?** In production, a new airline (or a new negative reason) may appear. We do not want the code to crash.

Step 4 — Combine into a ColumnTransformer

- **HINTS:**

- from sklearn.compose import ColumnTransformer.
- It takes a list of tuples: (name, transformer, list_of_columns).

Step 5 — Add the model on top

- **HINTS:**
 - from sklearn.linear_model import LogisticRegression.
 - LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42, multi_class='multinomial').
 - Put it in a Pipeline with the preprocessor.
- **WHY class_weight='balanced'?** Only 16% of tweets are positive, 63% are negative. Without this, the model just predicts “negative” for everyone and gets 63% accuracy (but useless).
- **WHY multi_class='multinomial'?** We have 3 classes (negative, neutral, positive), not 2.

Step 6 — Train and evaluate

- **HINTS:**
 - full_pipeline.fit(X_train, y_train).
 - y_pred = full_pipeline.predict(X_test).
 - from sklearn.metrics import classification_report.
- **EXPECTED:** F1 on negative class around 0.80. F1 on neutral around 0.45 (neutral is hardest). F1 on positive around 0.55–0.65. Macro F1 around 0.60–0.65. Module 4 improves this.

Step 7 — Save the trained pipeline

- **HINTS:**
 - import joblib.
 - joblib.dump(full_pipeline, '/content/drive/MyDrive/airline_lab/airline_pipeline.joblib').

Phase C — Make ONE chart for James

James’s chart — “How many tweets did we route correctly?”

A confusion matrix with annotations and clear labels.

- **HINTS:**
 - Use ConfusionMatrixDisplay.
 - Or sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=[...], yticklabels=[...]).
- **Title:** “Routing accuracy: of 1,000 angry tweets, our baseline catches ~800.”
- **Takeaway for James:** “We catch most angry tweets correctly. We confuse neutral and positive sometimes. Marketing may get a few wrong tweets. Module 4 with a better model will fix this.”

Common mistakes in Class 5

Mistake	What goes wrong
Build the pipeline AFTER you scaled the data manually	Pipeline scales it twice. Numbers wrong.
Forget sparse_output=False in OneHotEncoder	Some downstream code breaks.
Use class_weight='balanced' AND oversampling	Over-correction. Pick one.
Forget random_state	Results change every run. Hard to debug.
Forget multi_class='multinomial'	Default uses one-vs-rest. For 3 classes that is OK but multinomial is better.

Self-check before Class 6

- ☐ Pipeline has preprocessor + model.
 - ☐ Preprocessor has numeric + categorical transformers.
 - ☐ `handle_unknown='ignore'` set.
 - ☐ `class_weight='balanced'` set.
 - ☐ Confusion matrix chart.
 - ☐ `airline_pipeline.joblib` saved to Drive.
-

Class 6 — End-to-End Lab (THE BIG ONE)

Scenario reminder: James is in the meeting room. He wants the FINAL clean dataset on his desk in 90 minutes. This is the lab.

Your goal

Take 1 raw CSV file. Produce ONE final .parquet file with the exact 19-column schema. Plus a 1-page findings report.

Time

90 minutes of focused work.

Inputs

- The raw CSV file `Tweets.csv` in your Drive folder

Outputs

- `airline_tweets_clean.parquet` (~14,000 rows × 19 columns) in Drive
 - `findings.md` (~1 page)
 - Your full Colab notebook
-

Phase A — Explore the data first (10 minutes)

You will reuse charts from Classes 1–5. Pick 3 of them. Put them all on ONE PAGE of your notebook with markdown headers: 1. Sentiment distribution (the 63% negative problem) 2. Top complaint topics (the routing list) 3. Top 10 most important features for prediction

These 3 charts tell James the whole story.

Phase B — Build the final dataset (70 minutes)

Required output schema

Your `airline_tweets_clean.parquet` MUST have these 19 columns, with these names and types:

#	Column	Type	Source
1	<code>tweet_id</code>	string	raw
2	<code>airline</code>	string	raw

#	Column	Type	Source
3	airline_sentiment_confidence	float	raw
4	negativereason	string (or "not_negative")	raw, filled
5	retweet_count	int	raw
6	tweet_year	int	engineered
7	tweet_month	int	engineered
8	tweet_dayofweek	int	engineered
9	tweet_hour	int	engineered
10	is_weekend	int (0/1)	engineered
11	text_length	int	engineered
12	n_mentions	int	engineered
13	n_hashtags	int	engineered
14	has_url	int (0/1)	engineered
15	n_caps_words	int	engineered
16	has_question_mark	int (0/1)	engineered
17	has_exclamation	int (0/1)	engineered
18	sentiment_lexicon_score	int	engineered
19	airline_sentiment	string (negative/neutral/positive)	TARGET (raw)

You should also keep the text column SEPARATELY (in a different file or as an extra column) for Module 7. But it is NOT part of the 19 required columns.

Lab phases (timed)

Phase	Time	What you do
1. Load	5 min	Load Tweets.csv. Confirm 14,640 rows.
2. Clean	15 min	Convert tweet_created to datetime (UTC). Drop 4 useless columns. Drop missing-text rows. Drop duplicates.
3. Date features	10 min	Engineer tweet_year/month/dayofweek/hour/is_weekend.
4. Text features	20 min	Engineer text_length, n_mentions, n_hashtags, has_url, n_caps_words, has_question_mark, has_exclamation.
5. Lexicon score	10 min	Build positive/negative word lists, compute sentiment_lexicon_score.
6. Fill missing + save	10 min	Fill negativereason with "not_negative". Validate schema. Save .parquet.
7. Findings	10 min	Write findings.md.

Total: 90 minutes.

Phase C — Findings report for James (10 minutes)

Write findings.md with these sections:

Final numbers

- Final row count: ____ (~14,000)
- Sentiment breakdown:
 - Negative: ____% (should be ~63%)
 - Neutral: ____% (should be ~21%)
 - Positive: ____% (should be ~16%)

Top 3 insights

1. _____
2. _____
3. _____

One question to investigate in Module 4

- _____

Suggested routing rules (for the simple rule-based first version)

- If `sentiment_lexicon_score` ≤ -3 AND `n_caps_words` ≥ 2 : route to a human agent.
- If `negativereason` == 'Late Flight': route to operations.
- If `negativereason` == 'Lost Luggage' or 'Damaged Luggage': route to baggage team.
- If `airline_sentiment` == 'positive': route to marketing.
- (You can refine these in the report.)

One chart that summarizes everything

Embed your most important chart. Suggestion: the bar chart of complaint topics by airline. James can see at a glance which airline has the worst late-flight problem.

Grading rubric (100 points)

Item	Points
All 19 columns exist with the right names and dtypes	25
Cleaning decisions written in markdown	10
Pipeline is reproducible (re-run from raw CSV in ONE command)	15
<code>sentiment_lexicon_score</code> computed (positive minus negative word counts, lowercased)	10
Sentiment distribution preserved (~63% / ~21% / ~16%)	10
At least 3 exploratory + 1 explanatory chart per class	15
<code>findings.md</code> has insights, not just numbers	10
Code is clean (comments, sensible variable names)	5

Submit

Upload to `module-3/class_6/submissions/<YourTeamName>/`: - `airline_tweets_clean.parquet` - Your Colab notebook (File > Download > Download .ipynb) - `findings.md`

Tips for the whole module

1. **Do NOT copy-paste code from this guide.** This guide gives you HINTS only. Write the code yourself. You will learn much faster.
2. **Save your notebook to Drive often.** Colab disconnects after 12 hours.
3. **Save intermediate .parquet files to Drive** (tweets_step1, tweets_step2, etc.). If something breaks, you do not redo everything.
4. **Pair-program.** One student types, one reads. Switch every 20 minutes.
5. **Make a chart BEFORE you write cleaning code.** You must SEE the problem before you can fix it.
6. **Make a chart AFTER each step.** Confirm your code did what you expected.
7. **Small data is your friend.** 14,000 rows runs in seconds. Use the speed: try many ideas, throw away the bad ones.
8. **Keep the text column safe somewhere.** Module 7 will need it. Do not lose it.
9. **Ask the mentor early.** If you are stuck for 20 minutes, ask. Do not waste an afternoon.

Good luck.